



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



MODULE 3

Semantics Analysis

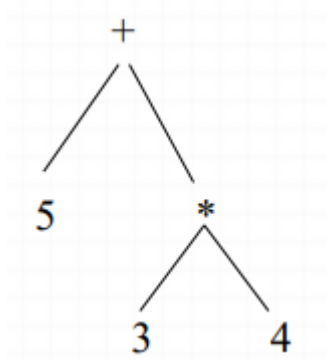
Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Applications of Syntax Directed Translation

- The primary application of Syntax Directed Translation is the **construction of syntax trees**.
- Compilers use syntax trees as an intermediate representation, using a common form of Syntax-Directed Definitions, the input string is converted into a tree.
- The compiler then traverses the tree using rules that are in effect an SDD on the syntax tree rather than the parse tree.
- Syntax-Tree
 - ***An intermediate representation of the compiler's input.***
 - ***A condensed form of the parse tree.***
 - ***Syntax tree shows the syntactic structure of the program while omitting irrelevant details.***
 - ***Operators and keywords are associated with the interior nodes.***
 - ***Chains of simple productions are collapsed.***
- In syntax tree for an expression
 - ***Each interior node represents an operator***
 - ***Children of the node represents operands of the operator.***

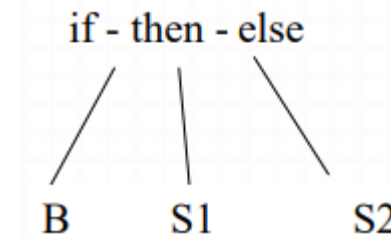
Expression

5 + 3 * 4



- Leaves: Identifiers or Constants
- Internal nodes: Labelled with operations
- Children: of a node are its operands

if B then S1 else S2



Statement:

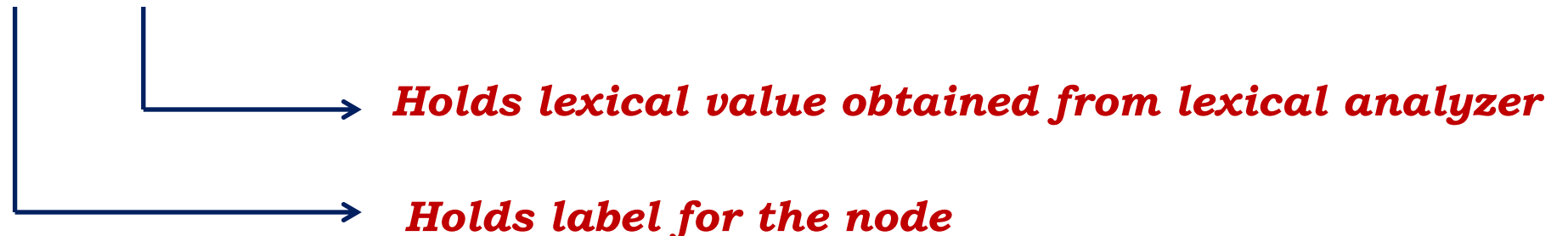
- Node's label indicates what kind of a statement it is
- Children of a node correspond to the components of the statement

- We consider two SDD's for constructing syntax trees for expressions.
 - First: An **S-attributed definition**, is suitable for use during bottom-up parsing.
 - Second:, **L-attributed definition** is suitable for use during top-down parsing.

1. Construction of Syntax Trees

- Each node in a syntax tree represents a structure, and its child nodes represent the key parts of that structure.
- A syntax-tree node for the expression $E_1 + E_2$ has label $+$ and has two children that represent the subexpressions E_1 and E_2
- *Syntax tree nodes can be created as objects with an appropriate number of fields.*
- Each **object** will have an **op** field that is the label of the node.
- The **objects** will have additional fields as follows:
 - (i) If the node is a leaf, an additional field holds the lexical value for the leaf.
 - A constructor function **Leaf (op, val)** creates a leaf object.
 - This is called only for terminals.

Leaf (op, val)



- If the **node is an interior node**, there are as many additional fields as the node has children in the syntax tree.
 - A constructor function Node takes two or more arguments:
 - **Node(op,c₁,c₂,c₃,...,c_k)** creates an object with first field op and k additional fields for the k children c₁,c₂,c₃,...,c_k

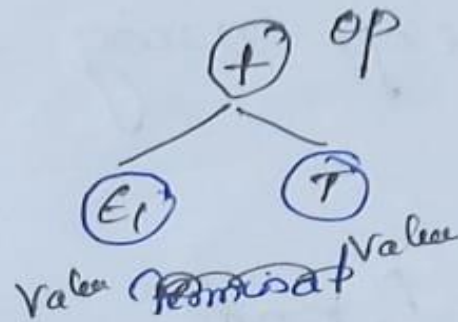
Node (op,c₁,c₂,c₃,...,c_k)



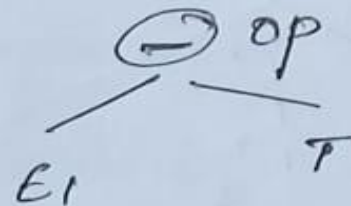
Creates node

$$E \rightarrow E_1 + T$$

$$E \rightarrow E_1 - T$$



Node (op, $\underline{G.w..(k)}$)
 $\downarrow \quad \downarrow \quad \downarrow$
 ('+', E1.node, T.node)



Node (-, G.node, T.node)

Single terminal

$$T \rightarrow id$$

$$T \rightarrow num$$

id is terminal
 $T(\text{root})$
 $\downarrow$
 id (child)
 (id.entry)

Leaf (op, val)
 $\downarrow \quad \downarrow$
 id id.entry

Example: S-Attributed

The S-attributed definition in table below constructs syntax trees for a simple expression grammar involving only the binary operators “+” and “-”

- These operators are at the same precedence level and are jointly left associative.
- Each nonterminal has a synthesized attribute called "node," which represents a syntax tree node.

- **First Production:** $E \rightarrow E_1 + T$ use, its corresponding rule that creates a node with + for op and **two children**, **$E_1.node$** , and **$T.node$**
- **Second production:** Have a similar rule as previous.
- **Third production:** No node is created, since $E.node$ is similar to $T.node$

PRODUCTION	SEMANTIC RULES
$E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
$E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow (E)$	$T.node = E.node$
$T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
$T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

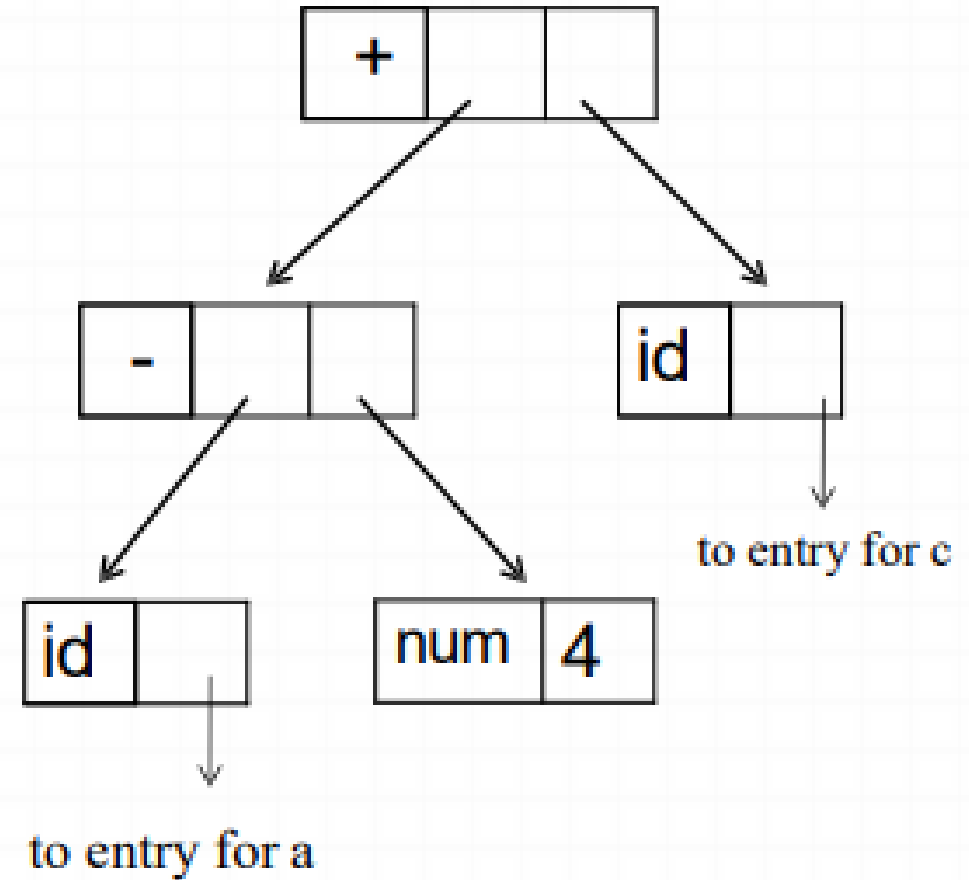
- **Fourth production:** No node is created. The value of **$T.node$** is same as **$E.node$** and since parenthesis are used only for grouping, they influence the structure of the parse tree and syntax tree.
- **Fifth and Sixth Production:** Have a single terminal to the right. The constructor Leaf is used to create a suitable node that will become the value of **$T.node$** .

Construct a syntax tree for the arithmetic expression and depict annotated parse tree for the expression

a - 4 + c

Steps in the construction of the syntax tree

- 1) $p_1 = \text{new Leaf}(\text{id}, \text{entry-a});$
- 2) $p_2 = \text{new Leaf}(\text{num}, 4);$
- 3) $p_3 = \text{new Node}('-', p_1, p_2);$
- 4) $p_4 = \text{new Leaf}(\text{id}, \text{entry-c});$
- 5) $p_5 = \text{new Node}('+', p_3, p_4);$

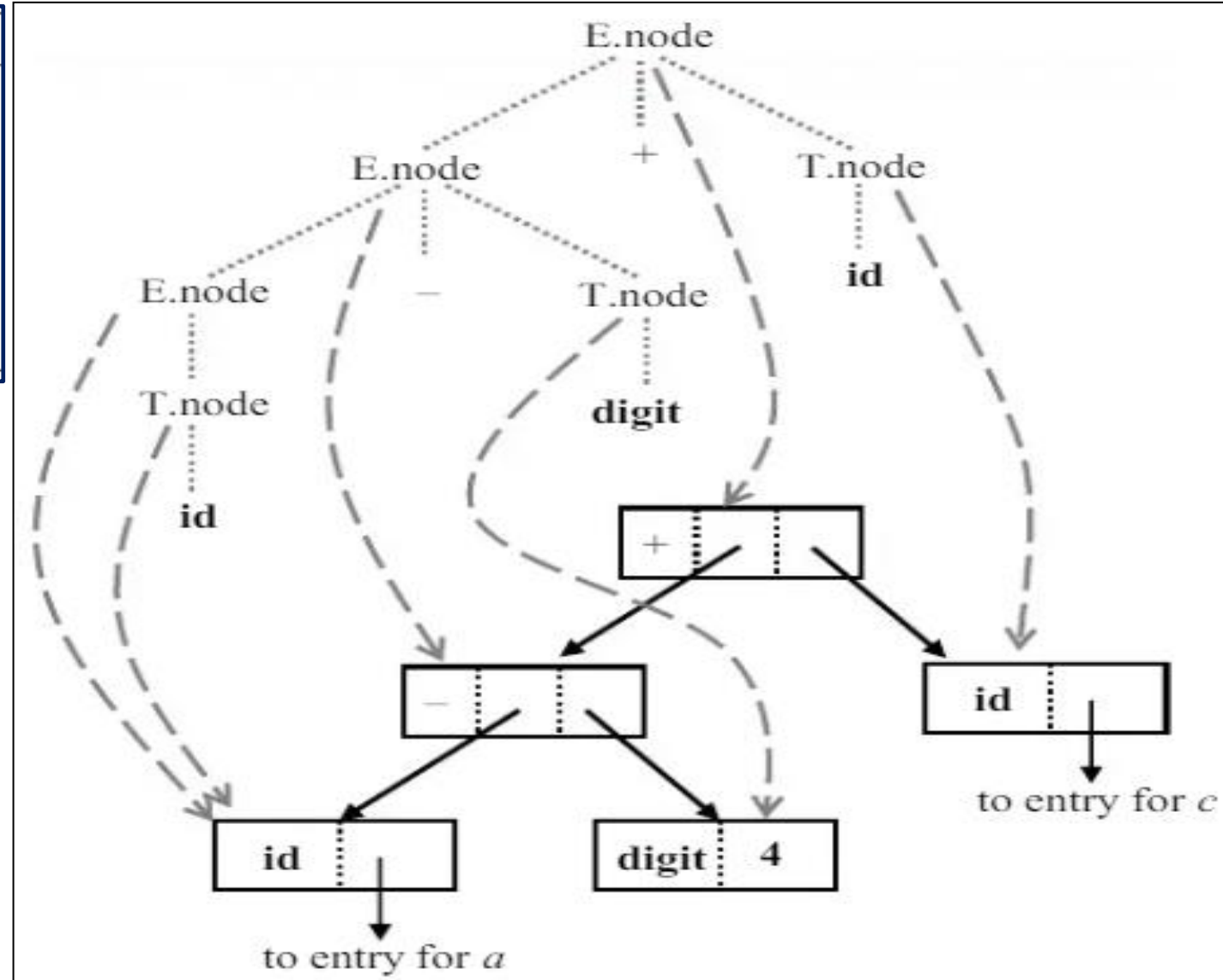


Annotated parse tree depicting construction of syntax tree

a - 4 + c

PRODUCTION	SEMANTIC RULES
$E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
$E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow (E)$	$T.node = E.node$
$T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
$T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

- The nodes of the syntax tree are shown as **records**, with the op field first.
- Syntax-tree edges are shown as **solid lines**.
- The primary parse tree, which doesn't need to be physically built, is represented with **dotted edges**.
- **Dashed line**, represents the values of E.node and T-node



Syntax tree for L-attributed definition

- The method of constructing syntax tree for L-attributed definition remains same as the method of constructing syntax tree for S-attributed definition.
- The functions **Leaf()** and **Node()** are used with same number and type of parameters

Example

Obtain the semantic rules to construct a syntax tree for simple arithmetic expressions using top-down approach with operators “+” and “-”

Solution:

- The grammar to generate an arithmetic expression with two operators + and - is

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow E - T \\ E &\rightarrow T \\ T &\rightarrow (E) \\ T &\rightarrow \text{id} \\ T &\rightarrow \text{number} \end{aligned}$$

- The given grammar isn't suitable for a top-down parser because it has **left recursion**. After removing the left recursion, the updated grammar is:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \\ E' &\rightarrow -TE' \\ E' &\rightarrow \epsilon \\ T &\rightarrow (E) \\ T &\rightarrow \text{id} \\ T &\rightarrow \text{number} \end{aligned}$$

- To construct syntax tree, first obtain the SDD for the above grammar.
- The SDD for the above grammar is

Production

Semantic Rule

$E \rightarrow T E'$

$E'.inh = T.val$

$E.val = E'.syn$

$E' \rightarrow + T E_1'$

$E_1'.inh = E'.inh + T.val$

$E'.syn = E_1'.syn$

$E' \rightarrow - T E_1'$

$E_1'.inh = E'.inh - T.val$

$E'.syn = E_1'.syn$

$E' \rightarrow \epsilon$

$E'.syn = E_1'.inh$

$T \rightarrow (E)$

$T.val = E.val$

$T \rightarrow \text{digit}$

$T.val = \text{digit.lexval}$

$T \rightarrow \text{id}$

$T.val = \text{id.entry}$

- Constructing syntax trees during top-down parsing can be obtained using the following semantic rules

Production

Semantic Rule

$E \rightarrow T E'$

$E'.inh = T.node$

$E.node = E'.syn$

$E' \rightarrow + T E_1'$

$E_1'.inh = \text{new Node}('+', E'.inh, T.node)$

$E'.syn = E_1'.syn$

$E' \rightarrow - T E_1'$

$E_1'.inh = \text{new Node}('-', E'.inh, T.node)$

$E'.syn = E_1'.syn$

$E' \rightarrow \epsilon$

$E'.syn = E_1'.inh$

$T \rightarrow (E)$

$T.node = E.node$

$T \rightarrow \text{digit}$

$T.node = \text{new Leaf}(\text{digit}, \text{digit.lexval})$

$T \rightarrow \text{id}$

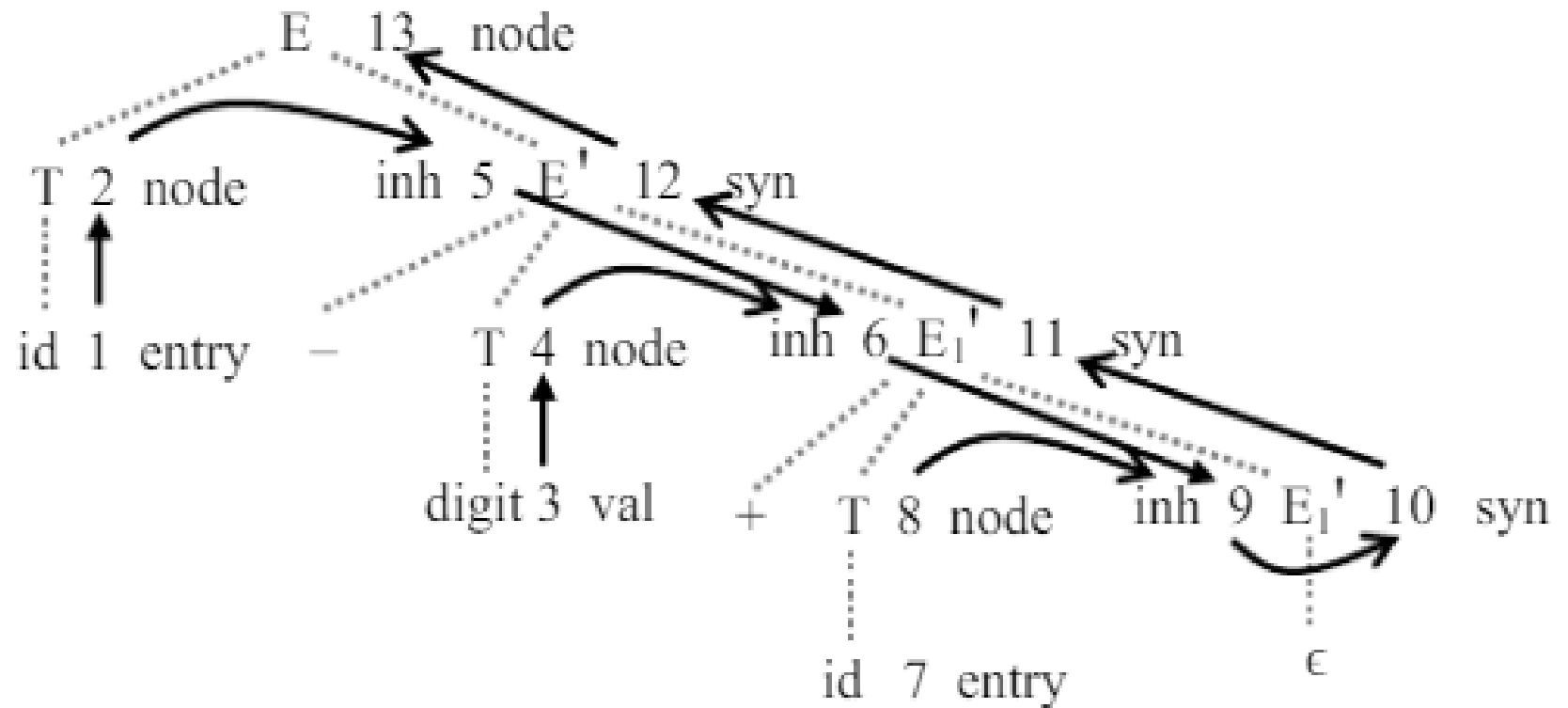
$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$

The **annotated parse tree** along with dependency graph for the expression

a - 4 + c

Steps

```
2) p1 == new Leaf(id, entry-a);  
4) p2 == new Leaf(num, 4);  
6) p3 == new Node('-', p1, p2);  
8) p4 == new Leaf(id, entry-c);  
9) p5 == new Node('+', p3, p4);
```



2. The Structure of a type

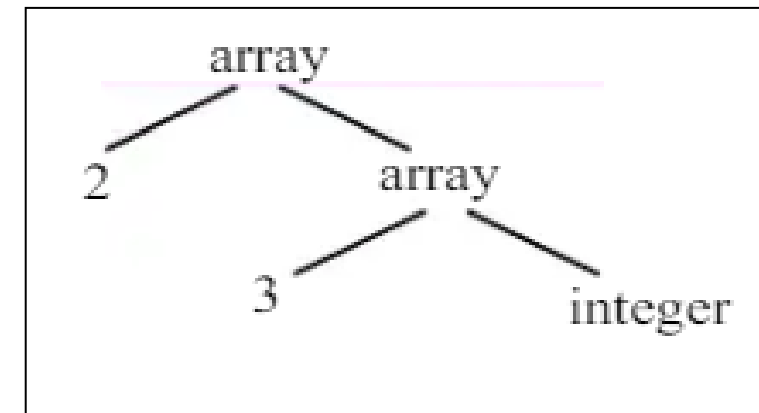
- Inherited attributes are useful when the structure of the parse tree differs from the abstract syntax of the input.
- Here we explain how inherited attributes can be used to carry information one part of the parse tree to another.

Example:

Give the syntax directed translation of type `int [2][3]` and also give the semantic rules for the respective productions.

Solution:

- The type `int [2][3]` can be read as, "array of 2 arrays of 3 integers"
- The corresponding type expression `array(2, array(3, integer))` is represented by the tree.
- The **operator** `array` takes two parameters, a **number** and a **type**.
- If types are represented by trees, then this operator returns a tree node labeled `array` with two children for a number and a type.



- The grammar for multi-dimensional array can be written as

$$\begin{aligned} T &\rightarrow BC \\ B &\rightarrow \text{int} \\ B &\rightarrow \text{float} \\ C &\rightarrow [\text{num}] C1 \\ C &\rightarrow \epsilon \end{aligned}$$

- The *non-terminal* T generates either a **basic type** such as **int** or **float**
- This is achieved using the following derivation

$$\begin{array}{ll} T \Rightarrow BC \Rightarrow \text{int } C \Rightarrow \text{int} & \text{[Finally replacing } C \text{ with } \epsilon] \\ \text{or} & \\ T \Rightarrow BC \Rightarrow \text{float } C \Rightarrow \text{float} & \text{[Finally replacing } C \text{ with } \epsilon] \end{array}$$

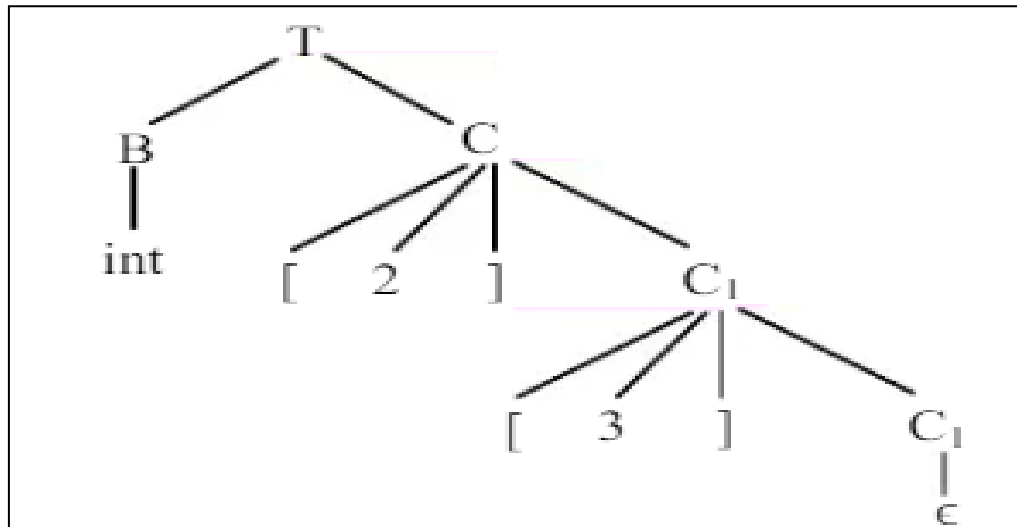
- The *non-terminal* T also generates **array of components** consisting of a sequence of integers where each integer is surrounded by brackets as shown below:

$$T \Rightarrow BC \Rightarrow \text{int } C \Rightarrow \text{int } [\text{num}] C \Rightarrow \text{int } [\text{num}] [\text{num}] C \Rightarrow \text{int } [\text{num}] [\text{num}]$$

- **Semantic rules**

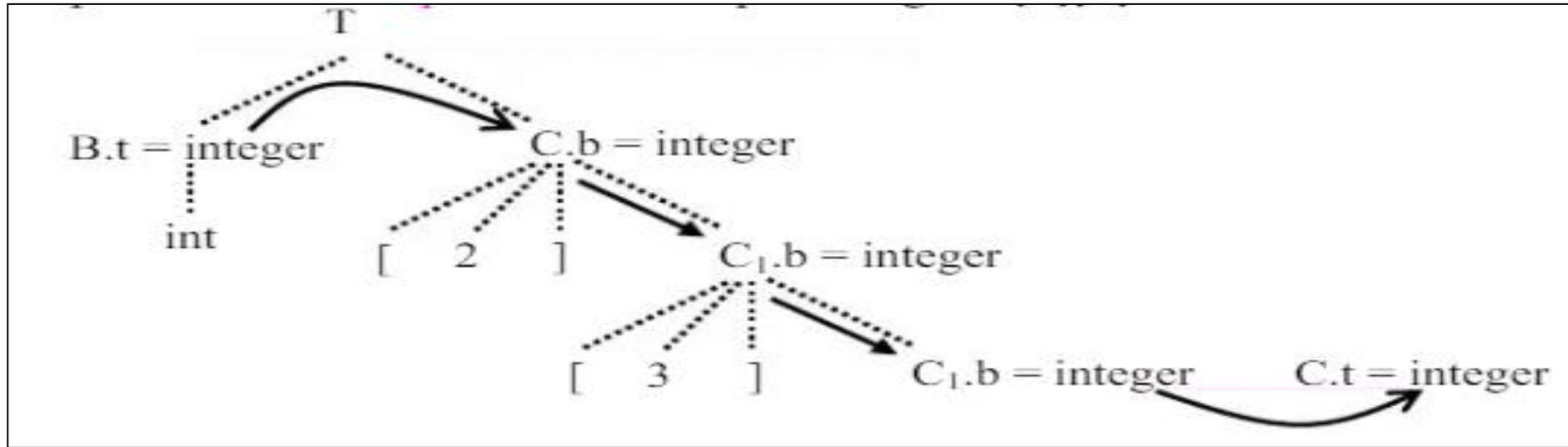
<u>Production</u>	<u>Semantic rule</u>	<u>Type</u>
$T \rightarrow B C$	$C.b = B.t$	Inherited
	$T.t = C.t$	Synthesized
$B \rightarrow \text{int}$	$B.t = \text{integer}$	Synthesized
$B \rightarrow \text{float}$	$B.t = \text{float}$	Synthesized
$C \rightarrow [\text{num}] C_1$	$C_1.b = C.b$	Inherited
	$C.t = \text{array}(\text{num.val}, C_1.t)$	Synthesized
$C \rightarrow \epsilon$	$C.t = C.b$	Synthesized

- The **parse tree** to get the string **int [2][3]**



- For the **synthesized attribute**, we use attribute name as ***t***
- For the **inherited attribute**, we use attribute name as ***b***

- The **partial annotated parse** for the input string **int [2][3]**



- The type **int** is moved to parent B and attribute t is synthesized.

<u>Production</u>	<u>Semantic rule</u>
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$

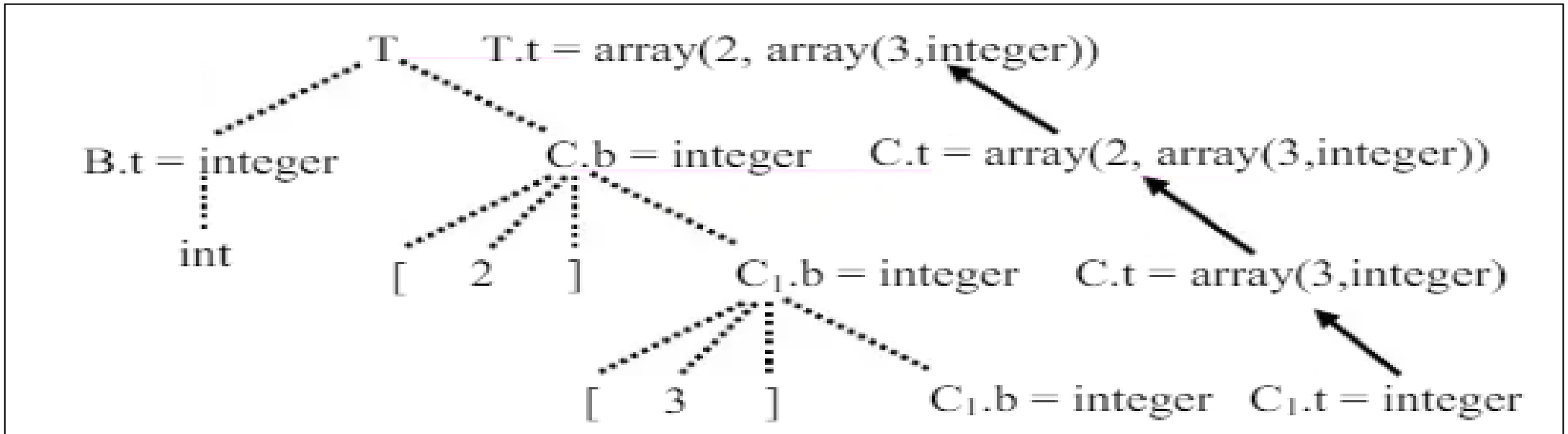
- The type **integer** has to be transferred from B.t on the left to C on the right using the production $T \rightarrow B C$ as shown using the arrow mark and hence it is inherited attribute denoted by b

<u>Production</u>	<u>Semantic rule</u>
$T \rightarrow B C$	$C.b = B.t$

- The **attribute value of C.b** moves down and copied into C_1 using the production $C \rightarrow [\text{num}] C_1$. It must be inherited.
- The production $C \rightarrow \epsilon$, the right most C in the above tree, takes its synthesized value $C.t$ from itself using inherited attribute value $C.b$
- The synthesized attribute value *integer* has to be moved from right most non-terminal to the root node

<u>Production</u>	<u>Semantic rule</u>
$C \rightarrow [\text{num}] C_1$	$C_1.b = C.b$

<u>Production</u>	<u>Semantic rule</u>
$C \rightarrow \epsilon$	$C.t = C.b$

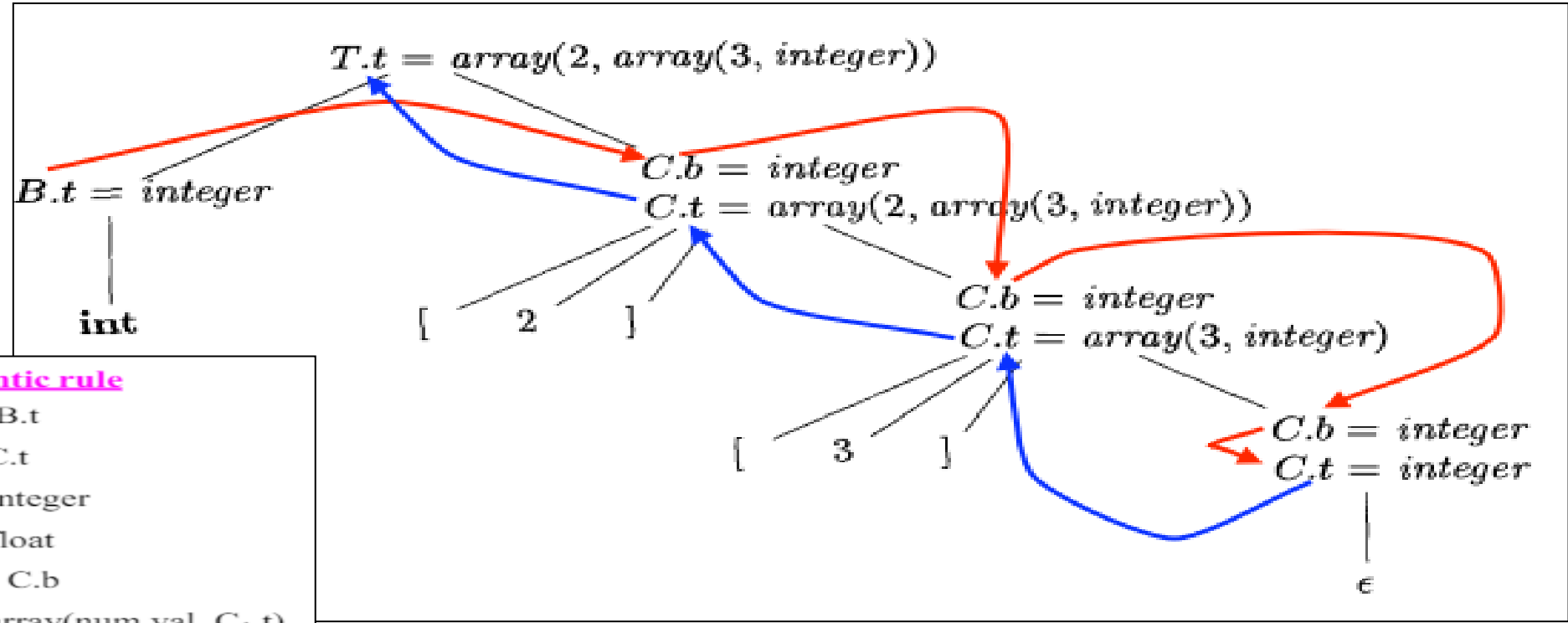


- Attribute value C1.t is moved to C.t upwards using the production $C \rightarrow [\text{num}] C$
- Finally, attribute value of C.t is moved to T.t using the production $T \rightarrow B C$

<u>Production</u>	<u>Semantic rule</u>
$C \rightarrow [\text{num}] C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$

<u>Production</u>	<u>Semantic rule</u>
$T \rightarrow B C$	$T.t = C.t$

The final annotated parse tree



<u>Production</u>	<u>Semantic rule</u>
$T \rightarrow B C$	$C.b = B.t$ $T.t = C.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] C_1$	$C_1.b = C.b$ $C.t = \text{array}(\text{num.val}, C_1.t)$
$C \rightarrow \epsilon$	$C.t = C.b$

Syntax Directed Translation Schemes

- A syntax directed translation is a context free grammar with **program fragments** embedded within **production bodies**.
- The **program fragments** are called **semantic actions** and can appear at any position within the production body.
- Translation schemes give a little bit information about implementation details.
- A translation scheme is a CFG where
 - **Attributes** are associated with **grammar symbols**
 - **Semantic actions** are inserted within **right sides of productions**
- Semantic actions are **enclosed within braces**.
- Semantic actions are treated as **terminal symbols**.



- All applications of SDD can be implemented using SDT.
- Translation Schemes deal with both **synthesized** and **inherited attributes**.
- Conceptually with both the syntax directed translation and translation scheme we
 - ***Parse the input token stream***
 - ***Build the parse tree***
 - ***Traverse the tree to evaluate the semantic rules at the parse tree nodes.***
- Any SDT can be implemented by first **building a parse tree** and then performing the actions in a **left-to-right depth-first order** i.e. during a pre-order traversal.
- Typically SDT's are implemented during parsing without building parse tree.
- The SDT's are used to implement two important classes of SDD's namely:
 - 1) Underlying grammar is LR-parsable and the SDD is S-attributed**
 - 2) Underlying grammar is LL-parsable and the SDD is L-attributed**
- During parsing, an action in a production body is executed as soon as all the grammar symbols to the left of action have been matched.

Example:

- **Translation Schemes for S-attributed Definitions**
- Production Semantic Rule

$$E \rightarrow E1 + T \quad == > \quad E.val = E1.val + T.val$$

A production of a syntax directed definition

$$E \rightarrow E1 + T \quad == > \{ E.val = E1.val + T.val \}$$

Production of the corresponding translation scheme

- **Translation Scheme with Inherited Attributes**

Production	Semantic Rules
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L1, id$	$L1.in = L.in;$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

$$\begin{aligned} D &\rightarrow T \{ L.in = T.type \} L \\ T &\rightarrow int \{ T.type = integer \} \\ T &\rightarrow real \{ T.type = real \} \\ L &\rightarrow \{ L1.in = L.in \} \\ &\quad L1, id \{ addtype(id.entry, L.in) \} \\ L &\rightarrow id \{ addtype(id.entry, L.in) \} \end{aligned}$$

The Applications of Syntax-Directed Translation Schemes are

- 1. Postfix Translation Schemes***
- 2. Parser-Stack Implementation of Postfix SDTs***
- 3. SDTs with Actions Inside Productions***
- 4. Eliminating Left Recursion from SDTs***
- 5. SDTs For L-Attributed Definitions***

1. Postfix Translation Schemes

- A Postfix Translation Scheme is a method used in ***syntax-directed translation***.
- It is based on a type of grammar called an **S-attributed grammar**, which can be parsed using a **bottom-up parser** (like an LR parser).
- In this scheme:
 - ***Each production rule of the grammar has semantic rules attached to it.***
 - ***These semantic rules are executed when the parser reduces the right-hand side of a production to the left-hand side non-terminal.***
- **Used to convert infix expressions into postfix notation**
- In postfix notation, the operator follows its operands, eliminating the need for parentheses and making it easier for machines to process mathematical expressions.

Example:

Create a postfix Syntax-Directed Translation (SDT) implementation for the desk calculator to evaluate the given expression.

SDT can be easily obtained SDD

Productions	Semantic Rules
$S \rightarrow E n$	$S.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.lexval}$

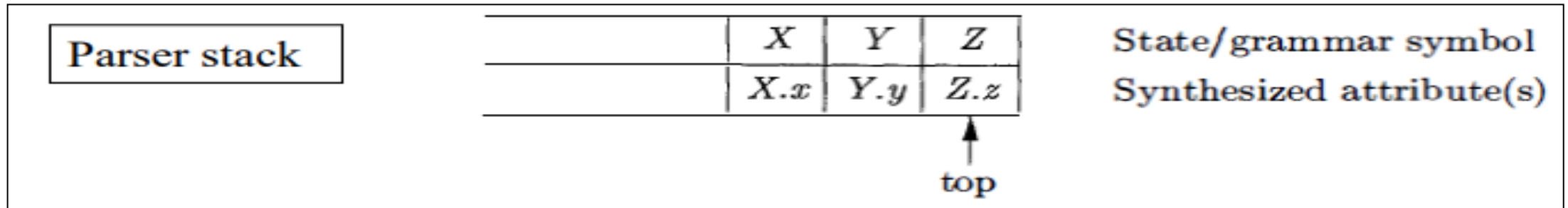
Postfix SDT implementing the desk calculator SDD

L	$\rightarrow$	$E n$	$\{ \text{print}(E.val); \}$
E	$\rightarrow$	$E_1 + T$	$\{ E.val = E_1.val + T.val; \}$
E	$\rightarrow$	T	$\{ E.val = T.val; \}$
T	$\rightarrow$	$T_1 * F$	$\{ T.val = T_1.val \times F.val; \}$
T	$\rightarrow$	F	$\{ T.val = F.val; \}$
F	$\rightarrow$	(E)	$\{ F.val = E.val; \}$
F	$\rightarrow$	digit	$\{ F.val = \text{digit.lexval}; \}$

val is a **synthesized attribute** for the non-terminals E, T, F
lexval is a **synthesized attribute** for the terminal digit

2. Parser-Stack Implementation of postfix SDTs

- Postfix SDT's can be implemented during **LR parsing** by executing the actions whenever reduction occurs.
- The **attribute of each grammar symbol can be stored on the stack**, making them accessible during the reduction process.



- The **parser stack** contains the **record for the non-terminals** in the grammar and their corresponding attributes.
- The non-terminal symbols of the production are pushed onto the parser stack.
- The **grammar symbols to be reduced to LHS of the production are present on top of the stack.**
- For example, for the production of the form: $A \rightarrow X Y Z$ and assume stack contains X , Y and Z and they are the symbols to be reduced.

- If the **attributes** are **synthesized** and **semantic actions** are at the **right ends** then attributes of the non-terminals are evaluated for the symbol in the top of the stack.
- When the reduction occurs at the top of the stack, the attributes are available in the stack, and after the action occurs these attributes are replaced by the corresponding LHS non-terminal and its attribute.
- Now, the LHS non-terminal and its attributes are at the top of the stack.

PRODUCTION	ACTIONS
$L \rightarrow E \mathbf{n}$	{ $\text{print}(\text{stack}[\text{top} - 1].\text{val});$ $\text{top} = \text{top} - 1;$ }
$E \rightarrow E_1 + T$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} + \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$E \rightarrow T$	
$T \rightarrow T_1 * F$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 2].\text{val} \times \text{stack}[\text{top}].\text{val};$ $\text{top} = \text{top} - 2;$ }
$T \rightarrow F$	
$F \rightarrow (E)$	{ $\text{stack}[\text{top} - 2].\text{val} = \text{stack}[\text{top} - 1].\text{val};$ $\text{top} = \text{top} - 2;$ }
$F \rightarrow \mathbf{digit}$	

Semantic Actions during Parsing

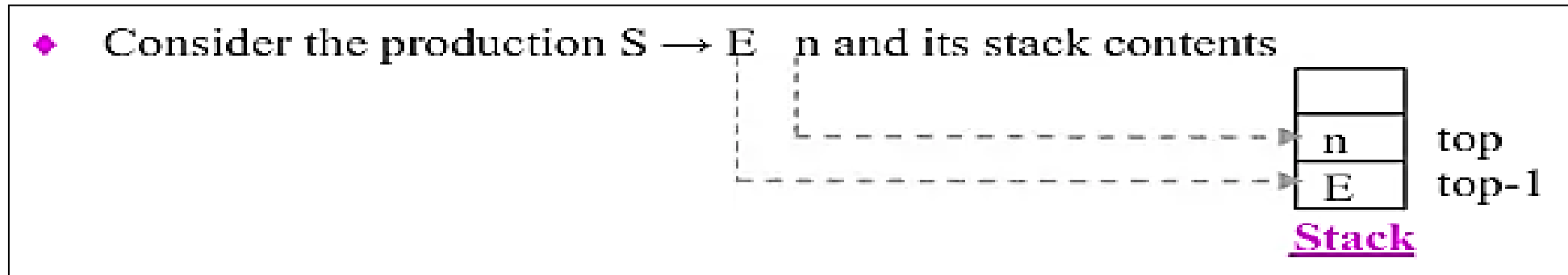
- When **shifting**
 - push the value of the terminal on the semantic stack
- When **reducing**
 - pop k values from the semantic stack, where k is the number of symbols on production's RHS
 - push the production's value on the semantic stack

Write the actions of desk calculator SDT so that they manipulate the parser explicitly.

The SDT for desk calculator can be written as shown below:

$S \rightarrow E n$	{ print(E.val) }
$E \rightarrow E_1 + T$	{ E.val = E ₁ .val + T.val }
$E \rightarrow T$	{ E.val = T.val }
$T \rightarrow T_1 * F$	{ T.val = T ₁ .val * F.val }
$T \rightarrow F$	{ T.val = F.val }
$F \rightarrow (E)$	{ F.val = E.val }
$F \rightarrow \text{digit}$	{ F.val = digit.lexval }

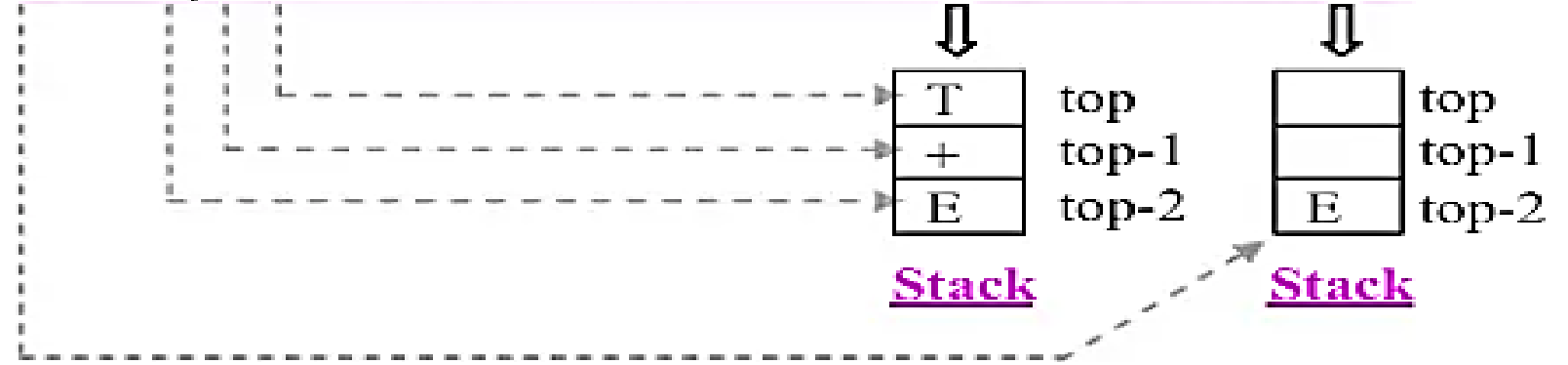
- The desk calculator can be implemented using the stack as shown below:



- The attribute value ***E.val*** which is in stack at position top-1 has to be printed.
- This can be done using the statement:

```
print(stack[top-1]);  
top = top-1
```

- Consider the production $E \rightarrow E_1 + T$ and its stack contents before and after reduction



- The equivalent statements are:

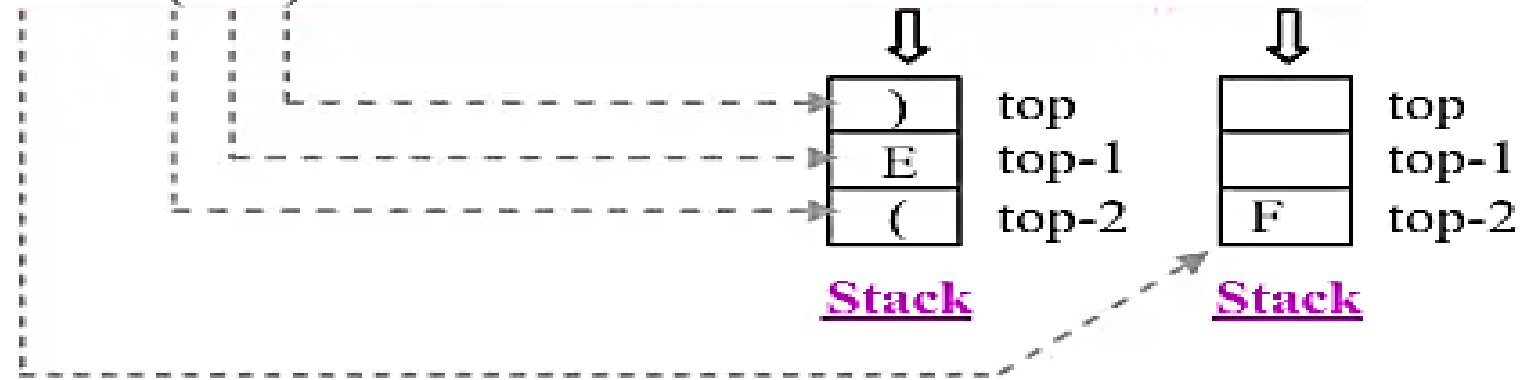
```
stack[top-2].val = stack[top].val + stack[top-2].val
top = top-2
```

- Similarly, for the production $T \rightarrow T_1 * F$, we can use the following statements:

```
stack[top-2].val = stack[top].val * stack[top-2].val
top = top-2
```

- For the productions $E \rightarrow T$ and $T \rightarrow F$, no action is necessary because the length of the stack will not change.

- ◆ Consider the production $F \rightarrow (E)$ and its stack contents before and after reduction



- The equivalent statements are:

$stack[top-2].val = stack[top-1].val$
 $top = top-2$

PRODUCTION	ACTIONS
$L \rightarrow E \mathbf{n}$	{ $print(stack[top - 1].val);$ $top = top - 1;$ }
$E \rightarrow E_1 + T$	{ $stack[top - 2].val = stack[top - 2].val + stack[top].val;$ $top = top - 2;$ }
$E \rightarrow T$	
$T \rightarrow T_1 * F$	{ $stack[top - 2].val = stack[top - 2].val \times stack[top].val;$ $top = top - 2;$ }
$T \rightarrow F$	
$F \rightarrow (E)$	{ $stack[top - 2].val = stack[top - 1].val;$ $top = top - 2;$ }
$F \rightarrow \mathbf{digit}$	

3. SDT's with Actions Inside Productions

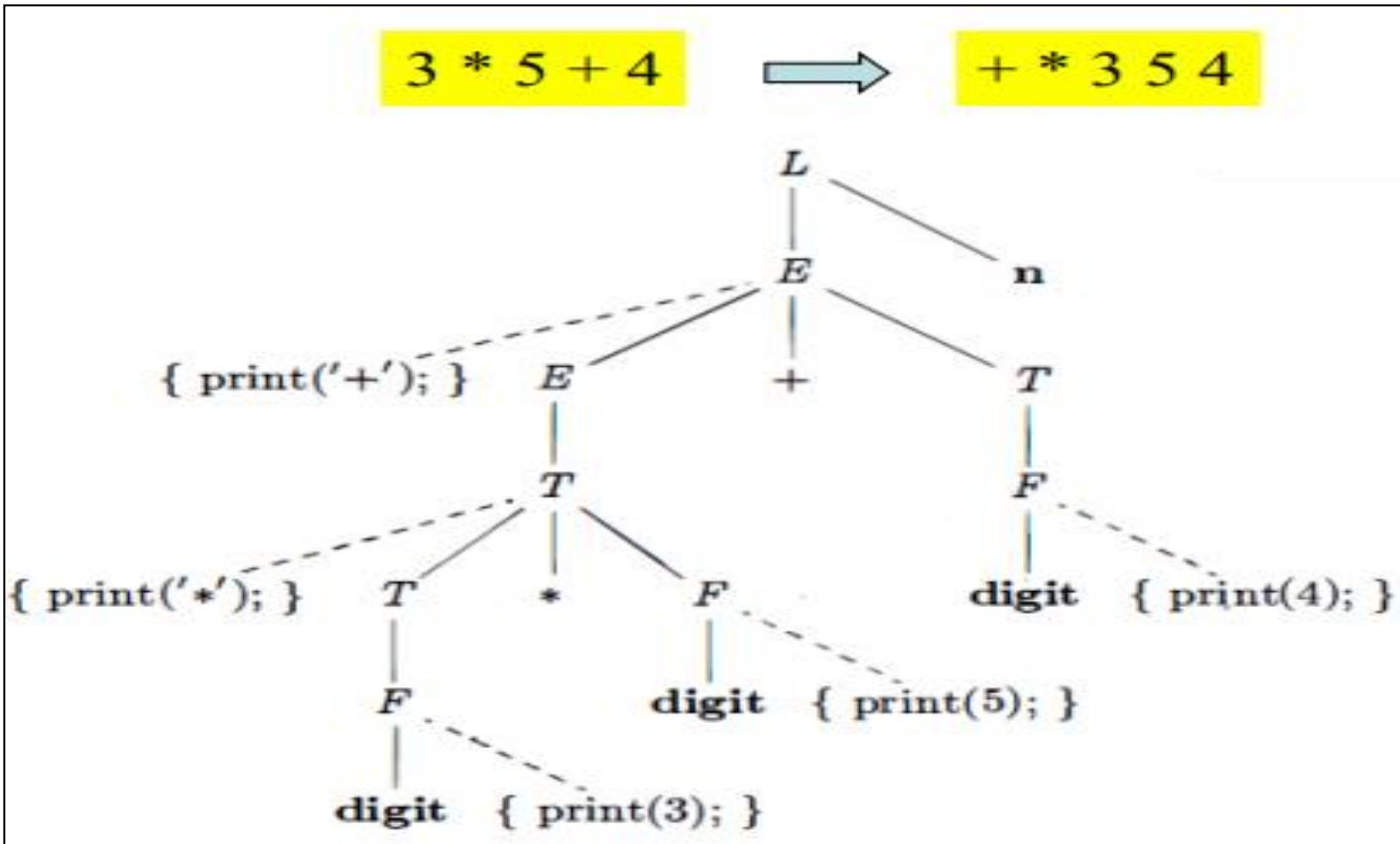
- When the *semantic actions are present anywhere on the right side of the production* then it is SDT with action inside the production.
- The action is performed immediately after all symbols to its left are processed.
- Given $B \rightarrow X \{ a \} Y$, an **action a** is done after
 - we have recognized **X** (if X is a terminal), or
 - all terminals derived from **X** (if X is a nonterminal)
- If the SDT is parsed in a **bottom-up parser** then, actions are performed immediately after the occurrence of a non-terminal at the top of the parser stack.
- If the SDT is parsed in a **top-down parser** then, actions are before the expansion of the non-terminal or if the terminal checks for input.

$S \rightarrow A + \{ \text{print '+'} \} B$
 $A \rightarrow \{ \text{print 'num'} \} B$
 $B \rightarrow \text{num} \{ \text{print 'num'} \}$

- Unfortunately, it's not feasible to implement this SDT during top-down or bottom-up parsing.
- The parser would need to print operators like * or + too early, before it can determine if they actually appear in the input expression.

Parse Tree With Actions Embedded

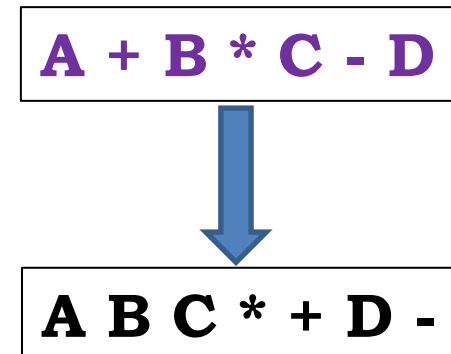
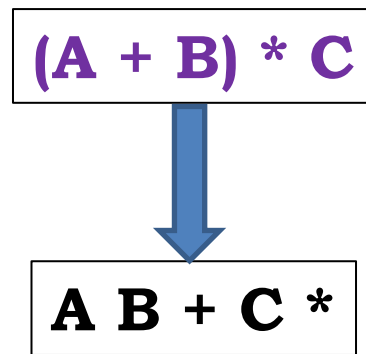
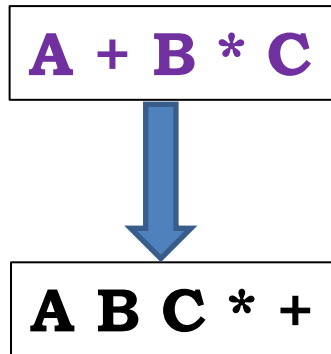
- Any SDT can be implemented as follows
 - Ignore the actions, parse input and produce a parse tree
 - Examine each interior node N and add actions as new children at the correct position.
 - Perform a preorder traversal and execute actions when their nodes are visited.



- 1) $L \rightarrow E n$
- 2) $E \rightarrow \{ \text{print}('+'); \} E1 + T$
- 3) $E \rightarrow T$
- 4) $T \rightarrow \{ \text{print}('*'); \} T1 * F$
- 5) $T \rightarrow F$
- 6) $F \rightarrow (E)$
- 7) $F \rightarrow \text{digit} \{ \text{print}(\text{digit.lexval}); \}$

Steps for Infix to Postfix Conversion

- 1. Operands:** Placed directly into the output (e.g., variables or constants).
- 2. Operators:** Placed into a stack based on their precedence and associativity.
- 3. Parentheses:**
 - Left parentheses (are pushed onto the stack.
 - When a right parenthesis) is encountered, operators are popped from the stack and added to the output until a left parenthesis is encountered and discarded.
- 4. At the end of the expression, all remaining operators in the stack are popped to the output.**

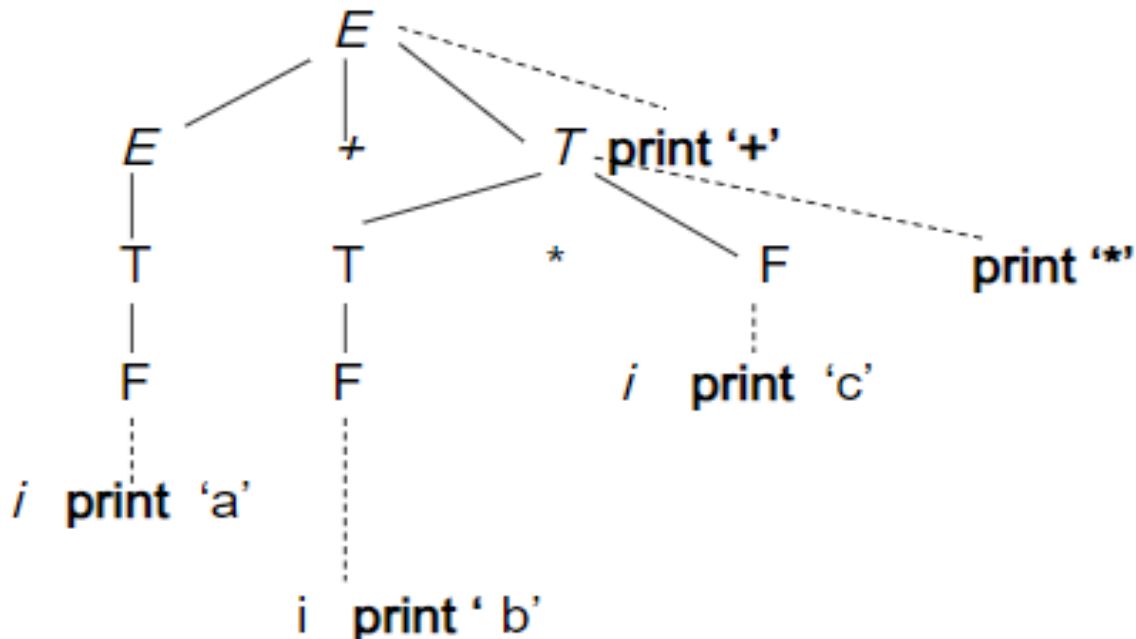


- The following grammar generates expressions over + and , while the SDT translates them into postfix.

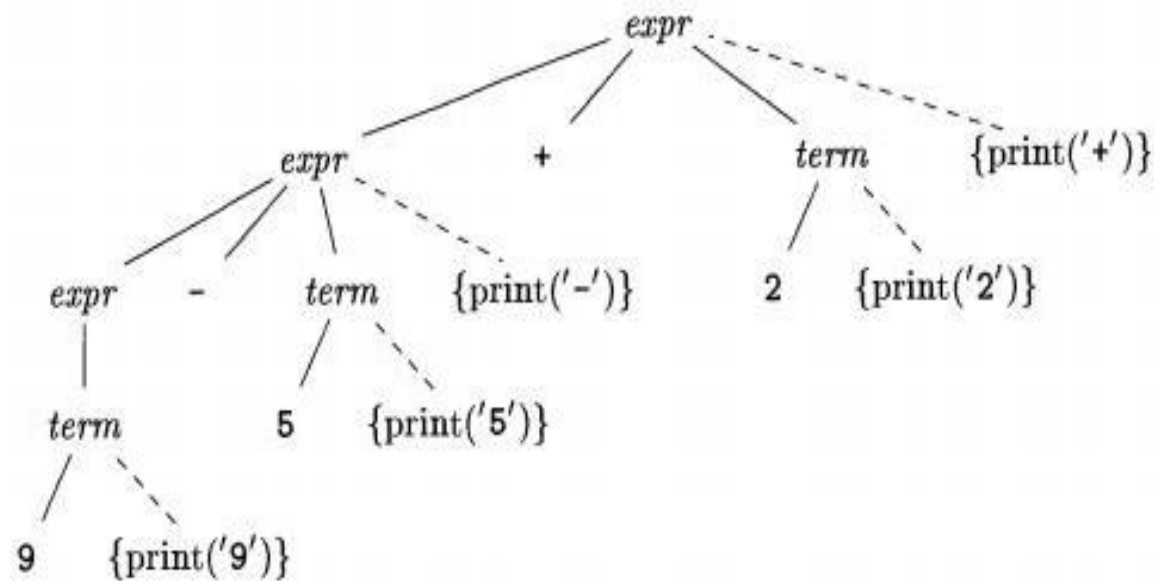
$E \rightarrow E + T \quad \{\text{print `+'}\}$
 $E \rightarrow T \quad \{\}$
 $T \rightarrow T * F \quad \{\text{print `*'}\}$
 $T \rightarrow F \quad \{\}$
 $F \rightarrow (E) \quad \{\}$
 $F \rightarrow \text{id} \quad \{\text{print (id.lexval)}\}$

$\text{expr} \rightarrow \text{expr}_1 + \text{term} \quad \{\text{print('+'})\}$
 $\text{expr} \rightarrow \text{expr}_1 - \text{term} \quad \{\text{print('-')} \}$
 $\text{expr} \rightarrow \text{term}$
 $\text{term} \rightarrow 0 \quad \{\text{print('0')}\}$
 $\text{term} \rightarrow 1 \quad \{\text{print('1')}\}$
 $\dots$
 $\text{term} \rightarrow 9 \quad \{\text{print('9')}\}$

Input : Translating a+b*c into abc*+



Input : Translating 9-5+2 into 95-2+

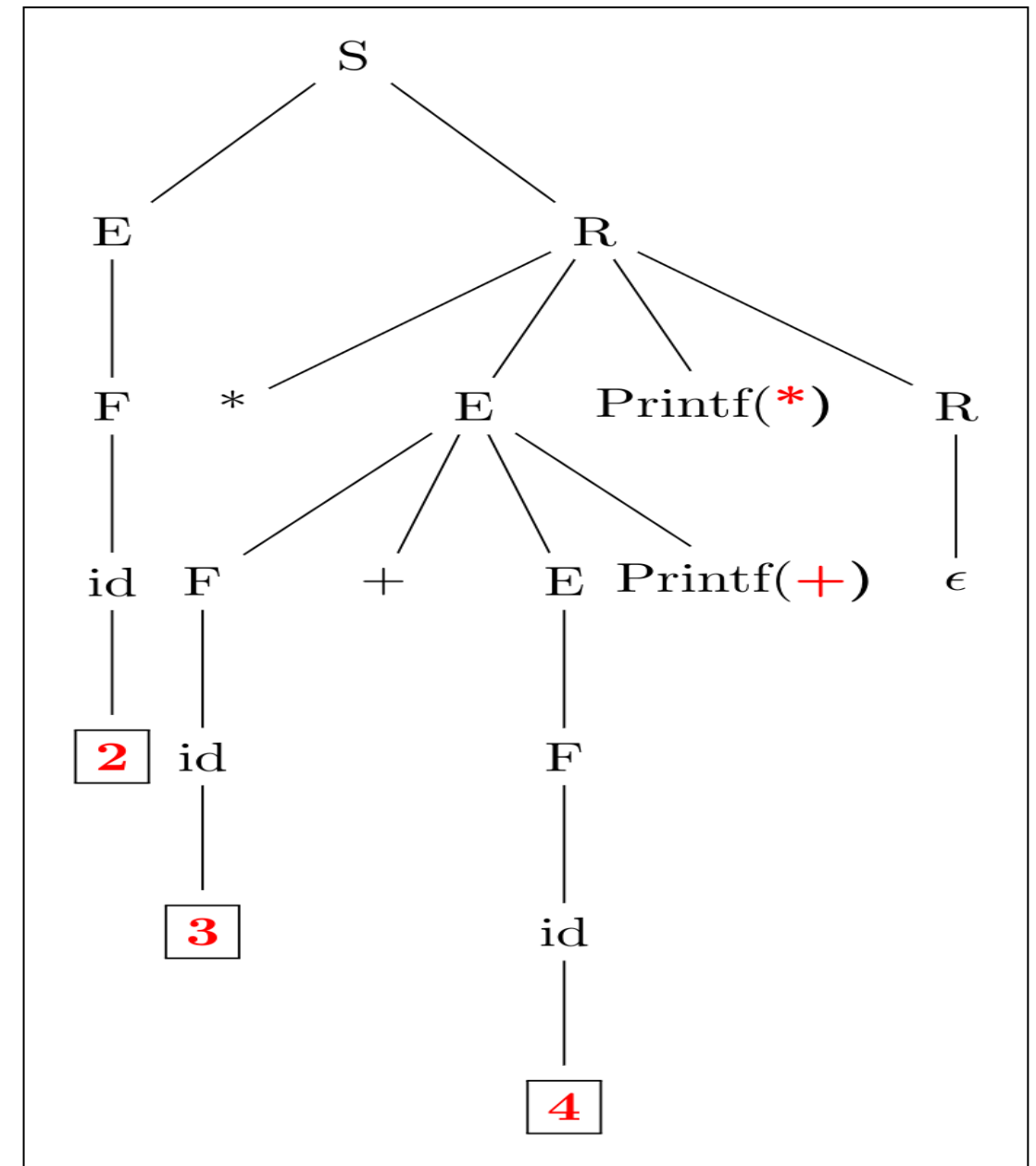


- Consider the following translation scheme.

$S \rightarrow ER$
 $R \rightarrow *E\{\text{print}('*');\}R | \epsilon$
 $E \rightarrow F + E\{\text{print}('+');\} | F$
 $F \rightarrow (S) | \text{id}\{\text{print}(\text{id.value});\}$
 $\}$

Here `id` is a token that represents an integer and `id.value` represents the corresponding integer value.

For an input '2*3+4', make a tree and perform post order evaluation

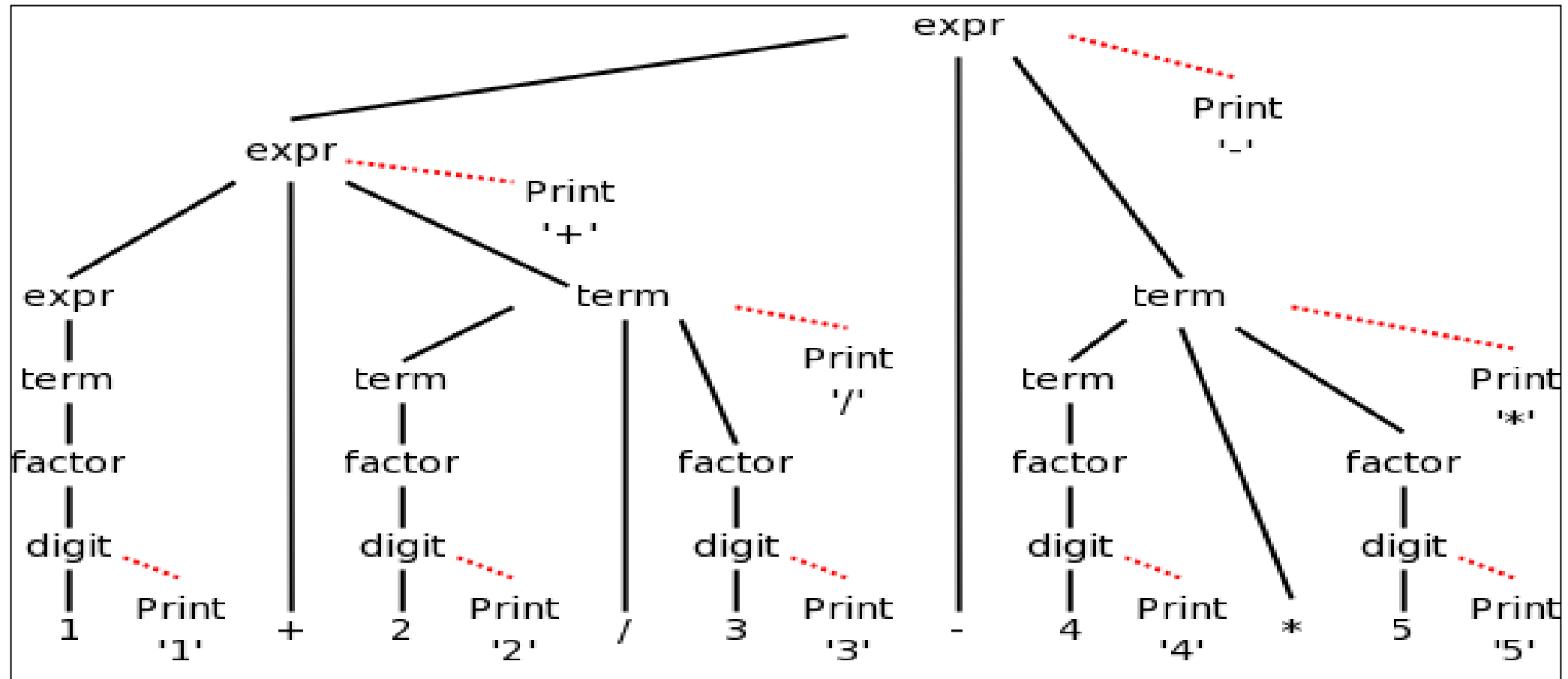


234+*

Example

expr $\rightarrow$ **expr** + **term** { **print**('+') }
expr $\rightarrow$ **expr** - **term** { **print**(' - ') }
term $\rightarrow$ **term** / **factor** { **print**(' / ') }
term $\rightarrow$ **factor** { **null** }
digit $\rightarrow$ 3 { **print**('3') }

1+2/3-4*5



4. Eliminating Left Recursion From SDT's

- The grammar with left recursion cannot be parsed by the top-down parser.
- In case of SDT we treat action as terminal symbol in the production.
- So, left recursion should be eliminated and the grammar can be transformed by eliminating it.
- Given

$E \rightarrow E1+ T \{ \text{print('+'); } \}$
 $E \rightarrow T$

$\alpha = + T \{ \text{print('+'); } \}$
 $\beta = T$

$E \rightarrow E + T \{ \text{print('+'); } \}$
 $E \rightarrow T$

$E \rightarrow T R$
 $R \rightarrow + T \{ \text{print('+'); } \} R$
 $R \rightarrow \epsilon$

$A \rightarrow A \alpha$
 $A \rightarrow \beta$



$A \rightarrow \beta R$
 $R \rightarrow \alpha R$
 $R \rightarrow \epsilon$

- Consider the following SDD with a single recursive production, single non recursive production, and a single attribute of the left-recursive nonterminal.
- Underlining grammar

$$\begin{array}{l} A \rightarrow A Y \\ A \rightarrow X \end{array}$$



$$\begin{array}{l} A \rightarrow X R \\ R \rightarrow Y R \\ R \rightarrow \varepsilon \end{array}$$

$$\begin{array}{l} A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \} \\ A \rightarrow X \{ A.a = f(X.x) \} \end{array}$$



$$\begin{array}{l} A \rightarrow X \{ R.i = f(X.x) \} \quad R \{ A.a = R.s \} \\ R \rightarrow Y \{ R_1.i = g(R.i, Y.y) \} \quad R_1 \{ R.s = R_1.s \} \\ R \rightarrow \varepsilon \{ R.s = R.i \} \end{array}$$

- X.a** is the **synthesized attribute** of left-recursive nonterminal **A**
- X** and **Y** are single **grammar symbols** with **synthesized attributes** **X.x** and **Y.y**
- An **arbitrary function** **g** compute **A.a** in the recursive production and an arbitrary function **f** computing **A.a** in the second production.
- The **inherited attribute** **R.i** is evaluated immediately before a use of **R** in the body.
- The **synthesized attributes** **A.a** and **R.s** are evaluated at the ends of the productions.

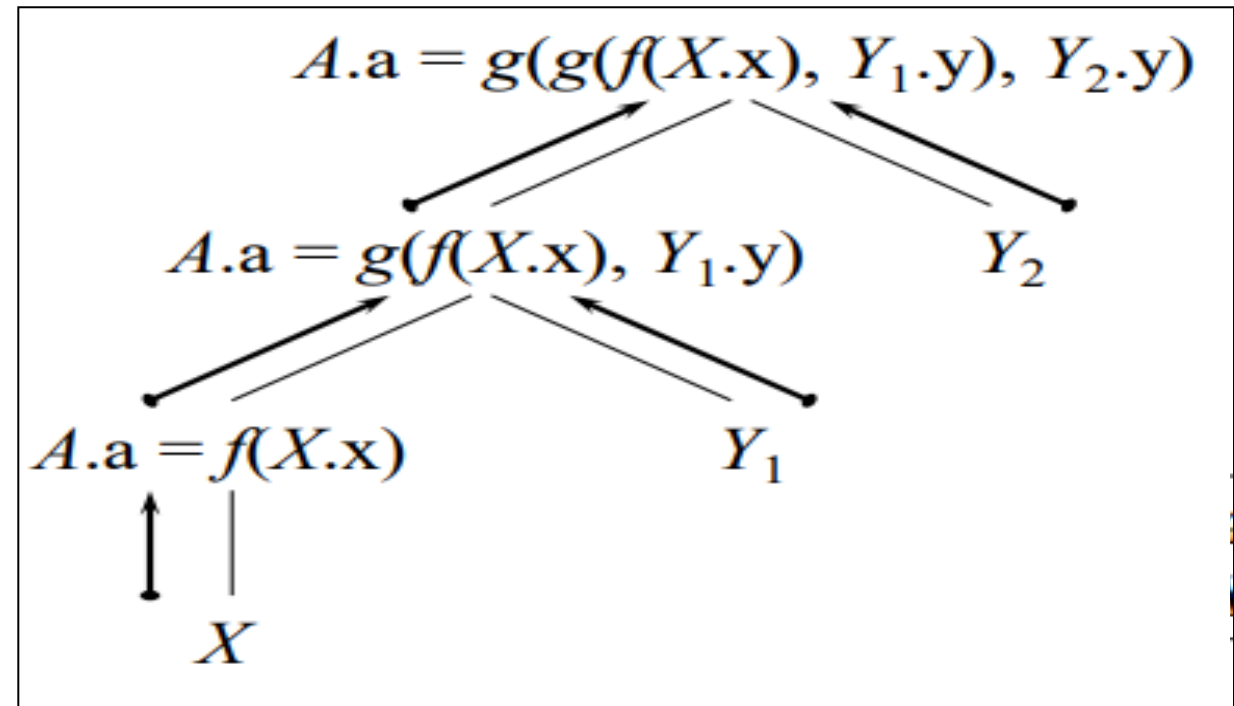
- Consider input string

$$\mathbf{XYY = XY_1Y_2}$$

$$\begin{array}{l} A \rightarrow A Y \\ A \rightarrow X \end{array}$$

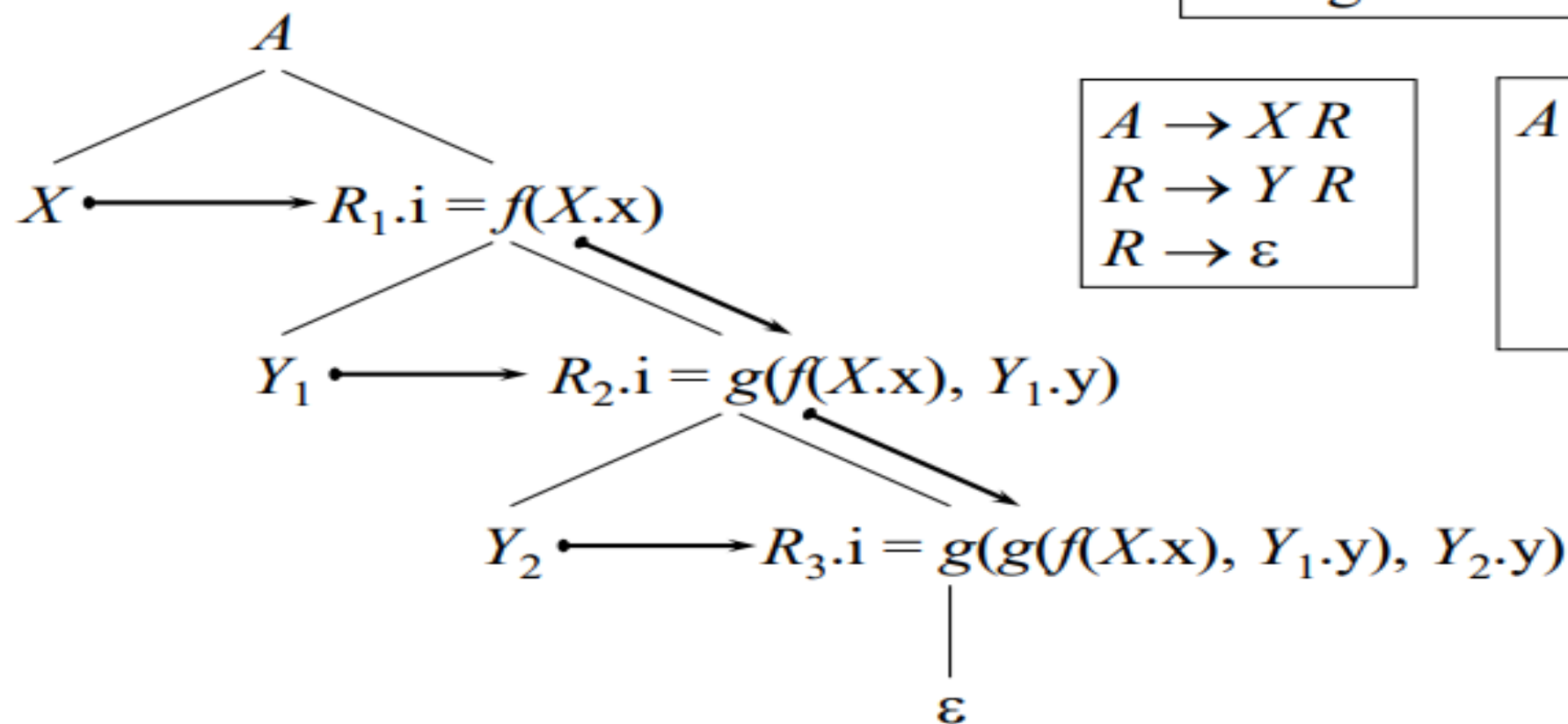
$$\begin{array}{l} A \Rightarrow A Y_2 \\ \Rightarrow A Y_1 Y_2 \\ \Rightarrow X Y_1 Y_2 \end{array}$$

$$\begin{array}{l} A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \} \\ A \rightarrow X \{ A.a = f(X.x) \} \end{array}$$



1. Flow of inherited attribute values

Consider the same input string $XY Y = XY_1 Y_2$



$A \rightarrow X R$
 $R \rightarrow Y R$
 $R \rightarrow \epsilon$

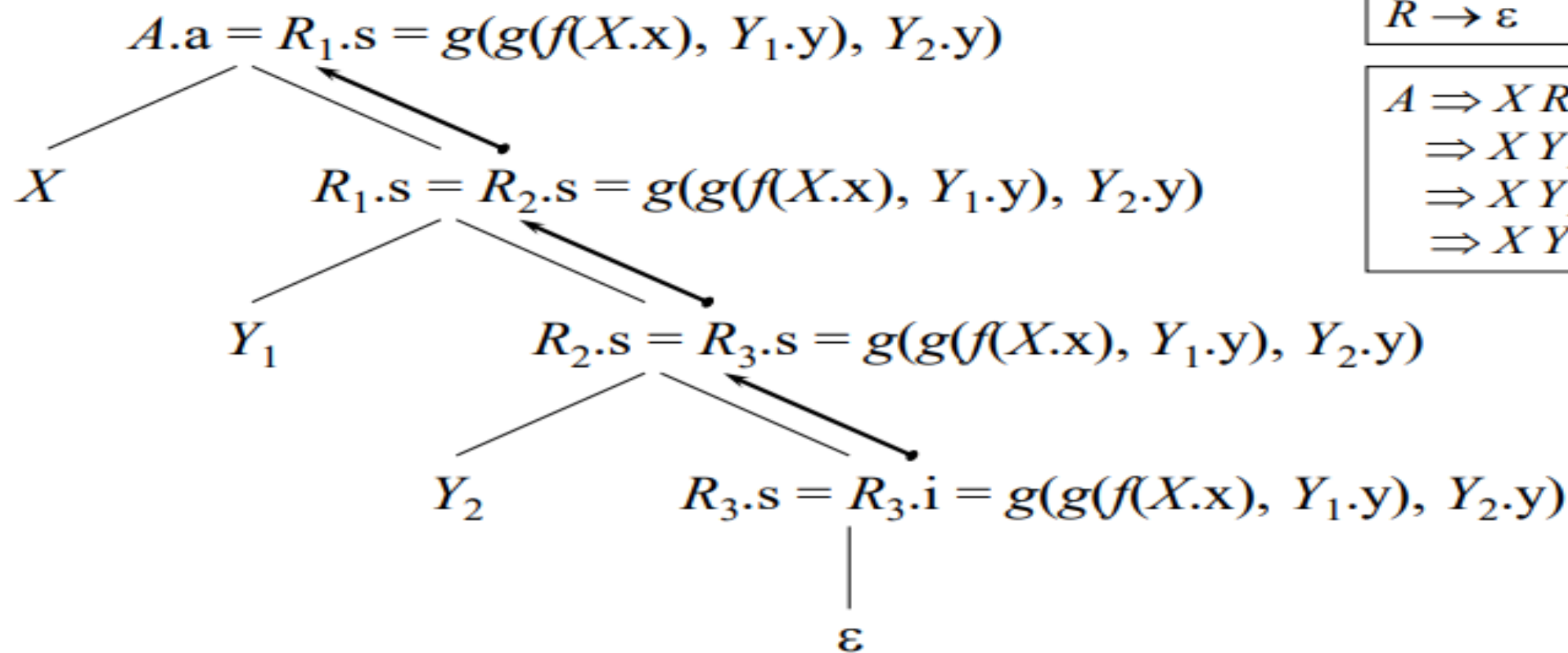
$A \Rightarrow X R$
 $\Rightarrow X Y_1 R$
 $\Rightarrow X Y_1 Y_2 R$
 $\Rightarrow X Y_1 Y_2$

$A \rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \}$
 $A \rightarrow X \{ A.a = f(X.x) \}$



$A \rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \}$
 $R \rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \}$
 $R \rightarrow \epsilon \{ R.s = R.i \}$

2. Flow of synthesized attribute values



$$\begin{aligned}
 A &\rightarrow X R \\
 R &\rightarrow Y R \\
 R &\rightarrow \epsilon
 \end{aligned}$$

$$\begin{aligned}
 A &\Rightarrow X R \\
 &\Rightarrow X Y_1 R \\
 &\Rightarrow X Y_1 Y_2 R \\
 &\Rightarrow X Y_1 Y_2
 \end{aligned}$$

$$\begin{aligned}
 A &\rightarrow A_1 Y \{ A.a = g(A_1.a, Y.y) \} \\
 A &\rightarrow X \{ A.a = f(X.x) \}
 \end{aligned}$$


$$\begin{aligned}
 A &\rightarrow X \{ R.i = f(X.x) \} R \{ A.a = R.s \} \\
 R &\rightarrow Y \{ R_1.i = g(R.i, Y.y) \} R_1 \{ R.s = R_1.s \} \\
 R &\rightarrow \epsilon \{ R.s = R.i \}
 \end{aligned}$$

Figure shows the [classic expression grammar](#) alongside a parse tree for $a \times 2 + a \times 2 \times b$.

Goal $\rightarrow$ *Expr*

Expr $\rightarrow$ *Expr* + *Term*

| *Expr* - *Term*

| *Term*

Term $\rightarrow$ *Term* $\times$ *Factor*

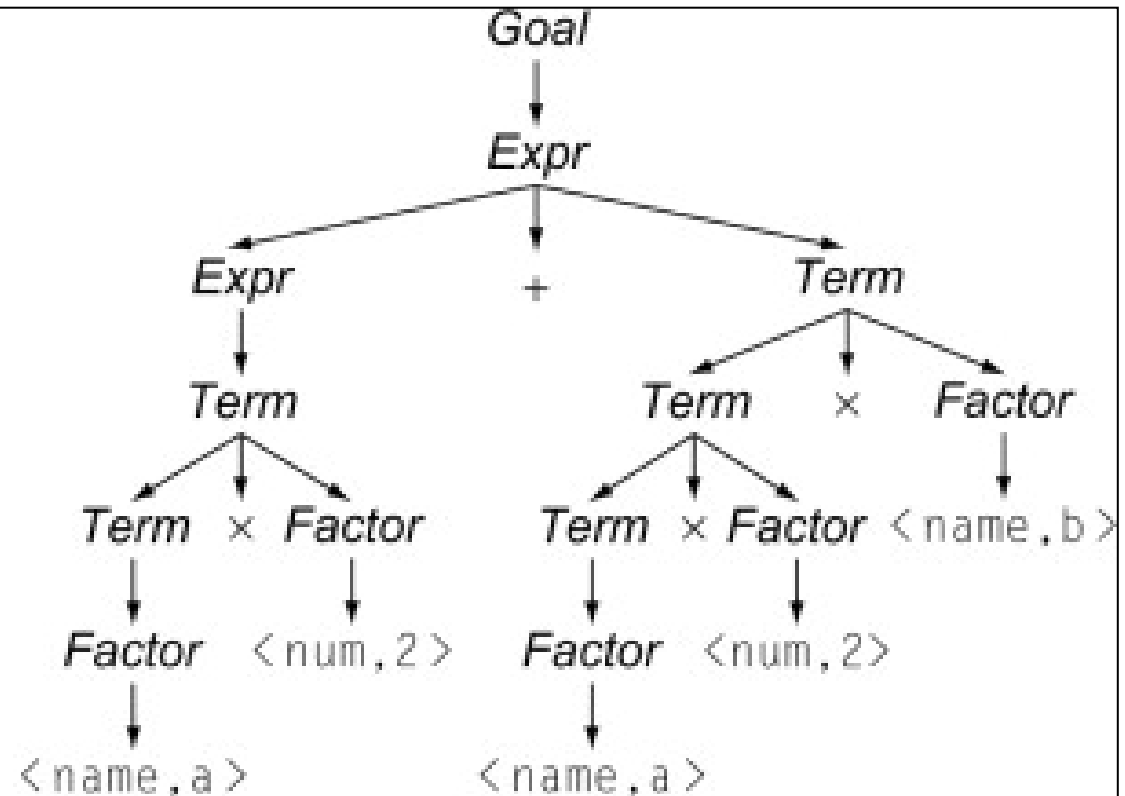
| *Term* $\div$ *Factor*

| *Factor*

Factor $\rightarrow$ (*Expr*)

| num

| name



(a) Classic Expression Grammar

(b) Parse Tree for $a \times 2 + a \times 2 \times b$

SDT for Infix to Postfix Translation

SDD

Production	Semantic Rules
$expr \rightarrow expr_1 + term$	$expr.code = expr_1.code term.code "+"$
$expr \rightarrow expr_1 - term$	$expr.code = expr_1.code term.code "-"$
$expr \rightarrow term$	$expr.code = term.code$
$term \rightarrow 0 1 \dots 9$	$term.code = "0"$ $term.code = "1"$... $term.code = "9"$

SDT

Production	Semantic Rules
$expr \rightarrow expr_1 + term$	$\{ print(" + ") \}$
$expr \rightarrow expr_1 - term$	$\{ print(" - ") \}$
$expr \rightarrow term$	
$term \rightarrow 0 1 \dots 9$	$\{ print("0") \}$ $\{ print("1") \}$... $\{ print("9") \}$

SDT Actions

